



Build your own `/generate` skill

Give your AI agent one command that creates images and videos on demand, always picks the cheapest capable model, never surprises you with a bill, and files every output in one place.



README

What this is. A build guide for a `/generate` skill: one command that makes images and videos, routes every request to the cheapest capable model, and files every output in a single folder.

How to set it up. Work through pages 2 to 6 in order. Page 3 gets the API keys, page 4 is the skill file itself, page 6 has the exact model ids and how each provider authenticates. Anything in a bordered box is copy-paste ready. Page 8 is a bonus prompt that builds a gallery for everything the skill makes.

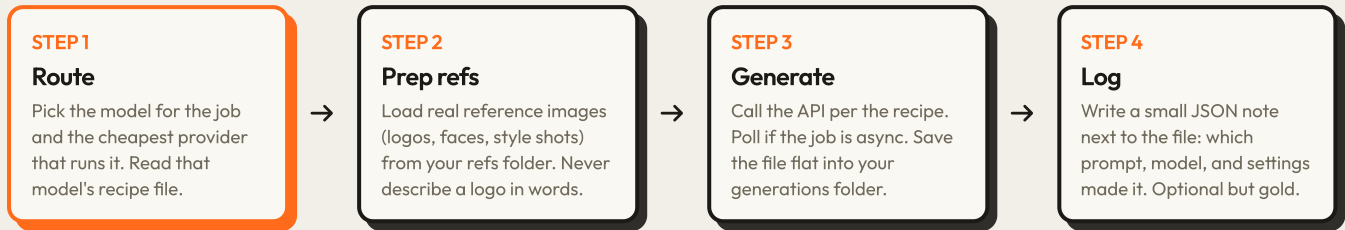
Two ways to get this skill. Build your own by following this guide, free. Or install the pre-built version: the exact `/generate` skill Jay runs daily, plus the Rubric Generations tab it feeds. That one comes with the RoboNuggets community at skool.com/robonuggets.

Works with Claude Code and any agent that reads skill files.

Every prompt in this guide is selectable, so you can copy it straight out of the PDF.

What you're building

A skill is a folder of markdown files your agent reads before it acts, like a playbook. Your /generate skill is a playbook for media: when you type "generate a thumbnail of X", the agent runs this four step pipeline.



The folder you'll end up with

```

your-workspace/
├── .claude/skills/generate/
│   ├── SKILL.md           the brain: routing table + rules (page 4)
│   └── models/
│       ├── image-model.md one recipe file per model (page 5)
│       └── video-model.md
├── generations/          every output lands here, flat, no subfolders
├── refs/                 logos, faces, style references you reuse
└── .env                  your API keys (page 3)
  
```

- a Why one flat folder?** Your whole media library lives in one place. Any gallery, dashboard, or script can read it with zero setup, including the bonus gallery on page 7. Your folder location is your choice, just keep it consistent.
- b Why recipe files per model?** Models change monthly. When a better one ships, you add one markdown file and your skill learns it. Nothing else changes.
- c Why rules in the skill?** Because the agent reads them every time. Cost gates and folder rules written once are enforced forever.

Get your **keys**

You don't need accounts with every AI lab. Aggregators host many models behind one key and one bill. Sign up for these three, grab an API key from each dashboard, and you can reach almost every image and video model that matters.

PROVIDER	SIGN UP AT	KEY NAME IN .ENV	WHY IT'S HERE
Kie AI	kie.ai	KIE_API_KEY	Often the cheapest route for popular models. Also has a handy file upload for turning local images into public URLs.
fal.ai	fal.ai	FAL_KEY	Fast, huge model catalog, the best docs of the three. Your first fallback.
WaveSpeed AI	wavespeed.ai	WAVESPEED_API_KEY	Broad catalog, useful when the other two lack a model or reject a job. Second fallback.

Store them in a .env file

```
KIE_API_KEY=your_key_here
FAL_KEY=your_key_here
WAVESPEED_API_KEY=your_key_here
```

Keep this file out of git (add `.env` to your `.gitignore`). Your skill will tell the agent to read keys from here, never to paste them into code.

What generations roughly cost

JOB	BALLPARK	NOTE
Draft image, cheap model	\$0.01 to \$0.03	Your default. Iterate freely here.
Quality image, top model	\$0.05 to \$0.15	For finals only, after you've picked a draft you like.
Video, per second	\$0.20 to \$0.35	A 10 second clip is \$2 to \$3.50. This is why cost gates exist.

Prices move fast, treat these as ballparks from the time of writing and check the provider's pricing page before you rely on them.

Write the **SKILL** file

Create `.claude/skills/generate/SKILL.md`. The frontmatter tells the agent when to use the skill, the body tells it how.

Copy this starter and swap in your models and folder.

COPY-PASTE · SKILL.MD STARTER

```

---
name: generate
description: Generate images and videos via AI model APIs. Triggers on
  /generate, generate image, generate video, create image, thumbnail,
  animate.
---

# /generate

## Models

| Task | Default model | Recipe |
|---|---|---|
| Image (default) | your cheap image model | models/image-model.md |
| Image (quality) | your hero image model | models/image-model-pro.md |
| Video (default) | your video model | models/video-model.md |

Read the recipe file before every generation.

## Provider routing
1. Default to the LOWEST COST provider that runs the model well
   (check Kie AI, fal.ai, WaveSpeed AI).
2. If the cheapest route lacks the model, fails auth, or errors,
   fall back to the next provider.
3. Never hide a provider swap. Say which route ran and why.

## Output
- Save every file FLAT into my generations folder: [your path here]
- No subfolders. Reference images live in generations/refs/
- Naming: {project}_{description}_{timestamp}.{ext}

## Rules
- Quote the cost and wait for my explicit go before any paid
  video run. One approval = one run.
- Draft on the cheap image model first. Only rerun on a quality
  model when I pick a favourite.
- Never describe a logo or face in text. Pass the real image
  file as a reference. If it's missing, stop and ask me for it.
- Run multiple generations one at a time to avoid rate limits.
- After every save, write the sidecar log (see Logging).
```

- 1 **The description line matters most.** It's how the agent decides to use the skill, so list the phrases you'll actually type.
- 2 **Fill in your folder path.** This differs for everyone. Pick a folder, write its full path into the Output section, and never move it.
- 3 **The rules section is the contract.** Everything on page 6 gets enforced from here.

One file per model

Each model gets a markdown file in `models/` holding everything the agent needs to call it: endpoint, auth, request shape, response handling. Open the model's API docs on your chosen provider and fill this template. Ten minutes per model, once.

COPY-PASTE · MODELS/YOUR-MODEL.MD TEMPLATE

Model Name

One line on what this model is best at and when to pick it.

```
| Field | Value |
|---|---|
| Model ID | the-exact-model-id |
| Provider | Kie AI / fal.ai / WaveSpeed AI |
| Method | Sync (instant reply) or Async (submit, then poll) |
| Type | Image or Video |
| API key | .env -> KEY_NAME |
| Docs | link to the provider's page for this model |
| Cost | rough price per image / per second |
```

Endpoint

POST https://...

Request format

(the exact JSON body from the docs, with prompt, aspect ratio, resolution, reference image fields)

Response handling

(where the image/video lives in the reply: base64 field, or a URL to download. For async: the status endpoint and the field that says "done".)

Notes

(gotchas: rate limits, max sizes, content rules, upload quirks)

The async pattern, since most video models use it

1. POST the job -> the reply contains a task id
2. Poll the status URL -> every 5 to 10 seconds, patiently
3. Status says complete -> the reply now contains a file URL
4. Download immediately -> result URLs often expire in hours
5. Save into your generations folder, then write the log

Start with two recipes: one cheap image model and one video model. Add more as you need them. Your skill grows one markdown file at a time.

Five worth wiring up

These five cover almost every job. Two for images, three for video. Each row below is everything you need to fill in a recipe file: the exact model id, which provider hosts it, and what it costs.

MODEL	BEST AT	PROVIDER	MODEL ID AND ROUGH COST
Nano Banana 2	Everyday images. Cheap, fast, strong with reference images.	Google AI Studio also on fal.ai	<code>gemini-3.1-flash-image-preview</code> Lite: <code>gemini-3.1-flash-lite-image</code> · about \$0.034 per 1K image
GPT Image 2	Text inside images. Signs, posters, menus, packaging, UI mockups.	fal.ai	<code>openai/gpt-image-2</code> Edit endpoint adds <code>/edit</code> · about \$0.05 per image at medium
Kling 3.0	General video. The sensible default: good motion, fair price.	Kie AI	<code>kling-3.0/video</code> std is 720p, pro is 1080p · 3 to 15 seconds
Veo 3.1	Higher quality video from a start frame. Slower.	Google AI Studio	<code>veo-3.1-generate-preview</code> 8 second clips, 720p, 1 to 4 samples per call
Seedance 2.0 Fast	Reference to video. Feed it up to 9 images and it animates them.	fal.ai	<code>bytedance/seedance-2.0/fast/reference-to-video</code> \$0.24 per second at 720p, so a 5 second clip is about \$1.21

Connecting: each provider authenticates differently

This is the single thing that trips people up. Same idea, three shapes. Get it right once per provider and every model on that provider works.

GOOGLE AI STUDIO

Key goes in the URL

```
POST .../models/
{model-id}:generateContent
?key={GOOGLE_KEY}
```

FAL.AI

Header, the word Key

```
POST https://fal.run/
{model-id}
Authorization: Key {FAL_KEY}
```

KIE AI

Header, the word Bearer

```
POST api.kie.ai/api/v1/
jobs/createTask
Authorization: Bearer {KIE_KEY}
```

Sync or async, per model

GPT Image 2, Nano Banana and Seedance reply with the finished result in one call. Kling and Veo hand you a job id, then you poll a status endpoint every 10 to 15 seconds until it says done. Write which one each model uses into its recipe file, because getting this backwards is the most common reason a first attempt appears to hang.

Don't wire all five on day one

START WITH THESE TWO

Nano Banana 2 Lite for images and **Kling 3.0** for video. Between them they handle the large majority of requests, they're the cheapest in their lane, and they teach you both patterns: one sync, one async. Two recipe files and your skill is genuinely useful.

ADD THESE LATER

Reach for **GPT Image 2** the first time you need readable text in an image. Add **Veo 3.1** when Kling's motion isn't good enough for a hero shot. Add **Seedance 2.0 Fast** when you want to animate reference images rather than a text prompt.

Model ids change when providers ship new versions. If a call returns a "model not found" error, open that provider's model page and copy the id fresh, then update the recipe file. That's the only maintenance this system needs.

Rails that save money

These five rules are already in the SKILL starter on page 4. Here's why each one earns its place.

Quote before video

Video is the expensive lane. The agent states model, duration, resolution, and expected dollars, then waits for your explicit go. Quoting alone is not approval, and one approval covers one run.

Draft cheap, finish pretty

Iterate on the \$0.01 model. Only when you pick a favourite does the same prompt rerun on a quality model. You stop paying premium prices for throwaway drafts.

Real refs, never described

A logo described in words comes back wrong every time: wrong shape, wrong colors, made-up details. Keep the real files in generations/refs/ and pass them into the API call.

One flat folder

Every output lands in the same folder, no subfolders. Sorting into folders feels tidy but breaks every tool that reads the library, and you'll never remember the sorting scheme anyway.

OPTIONAL • BUT DO IT

Log every prompt

Three weeks from now you'll look at an image and think "what prompt made THIS?". The sidecar log answers it. After every save, the agent writes a small JSON file next to the media, same name, .json extension.

```
HERO_THUMBNAIL_1774912000.JSON • SITS BESIDE
HERO_THUMBNAIL_1774912000.JPG

{
  "model": "your-image-model",
  "prompt": "the full text prompt that was sent to the API",
  "refs": ["refs/logo.png", "refs/headshot.jpg"],
  "params": { "aspect": "16:9", "size": "2K" },
  "created": "2026-07-30T09:41:00Z"
}
```

That's the whole feature. Same basename plus .json is the contract. Any future tool (or a plain folder search) can now recover exactly how any file in your library was made.

The gallery wall

Your generations folder is now filling up. This prompt turns it into a beautiful local gallery page: one file, no server, opens in your browser. Paste it into your agent exactly as written, it will ask about your folder path if it needs to.

COPY-PASTE · THE GALLERY PROMPT, VERBATIM

build me a single web page that shows every image and video my ai generates, all in one place, like a bento wall.

- it reads one folder on my computer called generations and shows everything sitting in it, newest at the top
- lay it out as a masonry wall - tiles keep their own shape, nothing gets cropped or squashed, rounded cards with even gaps, 4 columns wide and fewer as the window gets smaller
- videos start playing quietly when i hover over them and stop when i move away, images just sit there
- click any tile and it opens up big in the middle of the screen, click outside to close
- no search, no filters, no tabs, no side panels - just the wall of everything

keep it to one file so it runs by opening it, and make it feel like a proper finished gallery, not a rough draft.

when the page is done, add a line to my CLAUDE md so that from now on anything the /generate skill makes gets saved straight into that same generations folder, so it turns up on this page automatically. don't make the page pop open every time, just save the file there.

- 1 **Why this works:** the skill saves flat into one folder (page 4 rule), so the wall never needs updating. New generations just appear.
- 2 **The last paragraph is the trick.** It wires the rule into your agent's memory file, so the skill and the gallery stay pointed at the same folder forever.

WHERE THIS GOES NEXT

The easy part is generating a pretty picture. Anyone can do that by Friday. The part people and businesses actually pay for is the system around it: the routing that keeps costs sane, the gates that stop surprise bills, the library that never loses a prompt. That's what you just built, and it's a skill worth selling.

The pre-built version of everything in this guide is available ready to install: the exact `/generate` skill Jay runs every day, together with the Rubric Generations tab it feeds. It comes with the community, which is also where you'd go to learn to build and sell AI systems like this one, and to master Claude Code and Claude Design properly:

skool.com/robonuggets

RECAP

You now have

- 1 Three provider keys in a `.env` file, covering nearly every image and video model.
- 2 A **SKILL file** that routes every request to the cheapest capable model, with fallbacks.
- 3 A **recipe file per model**, so adding a new model is ten minutes of markdown.
- 4 **Guardrails**: cost quotes before video, cheap drafts first, real reference images, one flat folder.
- 5 A **sidecar log** beside every file, so no prompt is ever lost.
- 6 A **gallery wall** that shows everything your agent ever makes, automatically.

Type `/generate` and ask for something. That's the whole workflow now.